Where Does the Time Go When LLM Agents Work as a Team?
An Auditable Study of Scaling, Handoffs, and Self-Organized Crews

Anonymous Authors

Abstract

Teams of language-model agents are often assumed to get faster as they grow, but it is rarely measured how far a team is from the best any team could do, or where the lost time goes. We build a multi-agent environment in which a problem's total work and critical path are known exactly and cannot be shortcut by reasoning: a layered DAG of tasks, each of which can only be solved by evaluating a hidden keyed function one call at a time. This gives every run a hard lower bound T_{lb} and lets us account for each lost round from the traces. With teams of 1 to 32 identical agents and no prescribed organization, a frontier model reaches an 8.7× speed-up at 32 agents, but its efficiency against the bound falls from 0.98 alone to 0.49 at 16 agents. The loss concentrates in two places: handing a layer's answers to the next (4–5 rounds per layer, 2–3× a scripted team) and the final join, which costs one extra round for every answer its solver pulls from the shared store. When agents outnumber a layer's tasks, teams spontaneously split into two alternating crews that hide the handoff. We registered four predictions before running them: two quantitative scaling laws failed, while two mechanistic predictions held on every seed, including an out-of-sample shape and an intervention that forbids the second crew. [TODO: one sentence on the open-weight replication (Qwen3) once the runs are in.]

1 Introduction

Multi-agent systems built from large language models (LLMs) now divide software projects, research pipelines and simulations among many agents [4, 8, 14, 15, 20]. A natural question is how such a team scales: if one agent takes $T(1)$ to solve a problem, how long do N agents take? Recent work reports that adding agents helps on some tasks and hurts on others [10, 12, 16], and catalogues how multi-agent systems fail [3]. What is usually missing is a *reference point*: without knowing the fastest any team could solve a problem, a speed-up of 5× cannot be read as good or bad, and the lost time cannot be located.

Parallel computing has had this reference point for fifty years. For a computation with total work W and critical path (span) D, no schedule on N processors finishes in fewer than $\max(\lceil W/N \rceil, D)$ steps [2, 6]. We build an environment where this bound holds exactly for LLM agents, and use it to ask three questions. **(Q1)** How close do LLM teams come to the bound as they grow? **(Q2)** Where, step by step, does the remaining time go? **(Q3)** Do teams organize themselves to recover it, and can we show that causally?

[TODO: Figure 1 teaser: split-screen replay (web/compare.html, PR #2) of plain 32 agents (38 rounds) against no second crew (56 rounds); at round 30 half of the columns team's actions are waits. Export its SVG curve.]

Contributions.

- **An environment with exact bounds.** Tasks are chains of a hidden keyed hash with no repeated value, laid out as a layered DAG (§3). Reasoning cannot skip an evaluation, so total work and span are hard lower bounds for any strategy, and every accepted answer is audited call by call.
- **A time decomposition.** Synchronous lockstep rounds make a team's time comparable with the bound and with scripted reference teams, and split it into per-layer handoff loss and join loss (§4).
- **Findings.** LLM teams parallelize (8.7× at 32 agents) but lose 2–3× more per handoff than scripted teams; the join costs one round per answer read from the store; and teams with spare agents form two crews that halve the handoff loss (§5).
- **Pre-registered tests.** All four predictions were written before their runs (§6). We report the two that failed alongside the two that held, including an intervention that removes the second crew.

2 Related Work

LLM multi-agent systems. Frameworks organize agents by roles and conversation [11, 20], by standard operating procedures [8, 15], or by dynamic recruitment [4]; generative agents populate simulated towns [14]; debate improves answers by aggregating agents [5]. Deployed hierarchical systems report that hierarchies must be designed by hand and that details are lost in message passing [25]. These works measure task success; we measure *time against a bound* on a task whose success is fixed.

Table 1: Operations available to every agent.

operation	effect
`tasks`	list open, blocked and done tasks
`apply`	evaluate a task's hidden function once
`submit`	hand in an integer answer, checked exactly
`send`, `inbox`	message other agents
`put`, `get`, `keys`	shared, versioned store
`wait`	pass one round

Scaling the number of agents. Sampling-and-voting scales with agent count on reasoning benchmarks [12]; networks of hundreds of agents show a logistic quality curve [16]; and controlled comparisons of single- and multi-agent architectures find that the benefit depends on the task's decomposability [10]. Failure taxonomies attribute many multi-agent failures to inter-agent misalignment [3]. Our environment makes decomposability a parameter (width, depth and grain) and gives the exact best case, so the cost of coordination can be measured in rounds.

Parallel and pipelined LLM agents. A growing line of systems work makes agents run in parallel or in a pipeline by design. LLMCompiler plans a DAG of tool calls and dispatches independent ones at once [9]; DynTaskMAS and TIPEX schedule ready nodes of a dependency graph across agents [21, 23]; M1-Parallel races several agent teams [24]; and StreamMA streams each reasoning step downstream so that the next agent starts before the previous one finishes [22]. In all of these the schedule is engineered and success is measured in wall-clock latency on benchmarks. Closest to our question, Hogwild! Inference lets several workers share one KV cache and finds that they sometimes split the work without being told how [18]. We study the complementary case: no scheduler, agents that coordinate only through messages and a store, and a task whose optimal makespan is known. The two crews we observe are a pipeline that the agents build themselves, and we measure what it recovers against the bound.

Parallel performance models. Amdahl's law [1], Brent's bound [2, 6], bulk synchronous parallelism (BSP) [19] and the Universal Scalability Law (USL) [7] describe how parallel programs scale. We test whether the last two predict LLM teams; neither does.

3 Environment

The network. Agents act through a simulator that runs a task market, messaging, a public board and a versioned shared store, and charges every operation (Table 1). It never says how agents should cooperate.

The problem. A problem has *depth* d layers of *width* w tasks and one final task. Each task T owns a hidden function $f_T : \{0, \dots, 999\} \to \{0, \dots, 999\}$, a keyed BLAKE2 hash. A layer-0 task starts from a given number; a later task starts from the sum, mod 1000, of the answers of two random tasks in the layer before; the final task starts from the sum of the whole last layer. A task's answer is f_T applied *grain* g times in a row, and the only way to evaluate f_T is one `apply` call per value. The key is chosen so that no chain repeats a value, so no earlier output can stand in for a call.

Bounds. Solving a task costs exactly g applies and one submission, and it cannot start before its parents are known. With $n_T = wd+1$ tasks, total work is $W = n_T(g+1)$ actions and the span is $D = (d+1)(g+1)$, so a team of N needs at least

$$T_{\mathrm{lb}}(N) = \max\left(\lceil W/N \rceil,\ D\right) \tag{1}$$

rounds [2]. This holds for any strategy, however capable the model. An earlier version based on symbolic recurrences did not have this property: a strong model wrote the whole DAG as one closed-form expression without calling a tool. We report it as a negative result in Appendix A.

Teams. In the `peers` design, N identical agents start together with the same tools, the same briefing and an equal share of the call budget. They are told only that they are N identical agents working on one problem and should share the work as they see fit. There is no orchestrator or role. With $N = 1$, the same design is a single agent, so speed-ups compare like with like.

Time. LLM agents run at different speeds, so wall-clock time mixes model latency with coordination. We use *lock-step* rounds instead: in each round, every agent still working chooses one action, and the actions are then applied in a fixed order. A slow member therefore holds up the team, as a slow processor would. T is the round in which the final task is accepted.

4 Metrics

Speed-up is $T(1)/T(N)$; **efficiency** is $b = T_{\mathrm{lb}}/T \le 1$. As a practical reference, T_{ref} is the fastest of three scripted teams that broadcast each answer, share a store, or batch a layer's answers.

Where the time goes. For each layer $\ell \ge 1$ we compute the *handoff loss*: the rounds the layer took beyond the fastest it could go, given when its inputs became available. The *join loss* is the same quantity for the final task. Layer 0 has no handoff and is excluded. Both are computed from the traces, which also record every store read, message and claim.

Table 2: Scaling at grain 4 ($w = 16$, $d = 4$), means over 3 seeds.

N	T	speed-up	T_{lb}	T_{ref}	b
1	330.0	1.00	325	325	0.98
2	188.0	1.76	163	173	0.87
4	106.0	3.11	82	93	0.77
8	64.3	5.13	41	53	0.64
16	50.7	6.51	25	33	0.49
32	38.0	8.68	25	31	0.66

Audit. For every accepted answer we check its complete apply chain, its parents and its integer value. Across all runs reported here, every accepted node passed.

5 Results

Setup. The main problem is $w = 16$, $d = 4$ (65 tasks) at grain $g \in \{1, 4\}$, with $N \in \{1, 2, 4, 8, 16, 32\}$ and seeds 100–102. The model is a closed frontier model (`gpt-6.1-sol`, high reasoning effort) driven through its agent CLI. The study used about 520M tokens. [TODO: Qwen3-30B-A3B-FP8 [17] and a second open-weight model on one H100: same grid.]

5.1 Q1: Teams parallelize, but efficiency falls

Figure 1 and Table 2 show the scaling curve. A single agent is within 2% of the bound ($b = 0.98$ at grain 4, 0.96 at grain 1), which confirms that the problem leaves no room for shortcuts. Teams do parallelize: 32 agents solve the grain-4 problem in 38 rounds, an 8.7× speed-up over one agent. But efficiency falls with team size, to 0.64 at 8 agents and 0.49 at 16. Coarser tasks scale better: at 8 agents, b is 0.64 at grain 4 and 0.50 at grain 1, because each answer that must be shared carries more computation.

The curve is not monotone in efficiency. At 16 and 32 agents, the bound is the span ($T_{\text{lb}} = 25$), and 16 agents already have one agent per task. Yet 32 agents were faster than 16 on every seed (39, 38, 37 against 44, 58, 50). The rest of the paper explains this.

5.2 Q2: The time is lost at handoffs and at the join

Figure 2 decomposes the gap between T and the bound. Two mechanisms account for nearly all of it.

Handoff. Up to 16 agents, each layer after the first loses 4.1–5.1 rounds on average, against 1.2–1.6 for scripted teams. The traces show why: the agent that finishes a task and passes its answer on is also the agent that must then read the next layer's inputs and start the next task. Sharing, reading and starting are serialized in the same agent.

Join. The final task needs all 16 answers of the last layer. When its solver gathered them from messages, the join lost 1–4 rounds. In two of the three 16-agent runs, the solver instead read the answers from the store one key at a time, 15 and 6 reads, and the join lost 16 and 7 rounds. Across all 18 runs, the join's loss follows one round per store read on top of a base of 1–4 rounds. The three runs with the most reads (15, 15 and 6) lost exactly reads $+1$ (Figure 2b).

Together, these account for the gap between 16 and 32 agents. The per-layer gap is about 2.4 rounds over three layers, and the join gap is about 6 rounds, for a total of about 13 rounds against an observed 12.7.

5.3 Q3: Spare agents form two crews

The 32-agent teams split into two crews at the start of every run, without being told to: in the agents' own words, "a0–a15 handle L0 and L2; a16–a31 handle L1 and L3". While one crew computes a layer, the other reads the inputs it will need and waits to start, so the handoff overlaps with computation. Their layers lost 2.3–3.0 rounds, about half of what smaller teams lost.

Out of sample. We predicted, before running, that on a new 8-wide problem, 16 agents (two per task) would lose less per layer than 8 agents and would be faster on every seed. Both held on all three seeds (Figure 3a): 2.2 against 3.4 rounds per layer, and $T = 34.7$ against 38.7. On every seed the 16-agent teams again formed two crews, a0–a7 and a8–a15, on alternating layers.

Intervention. The association could be produced by something else that comes with having more agents. The `peers-columns` intervention adds one rule to the 32-agent team: agent a_K does column K in every layer, and agents without a column take no task. This is exactly the assignment that 16-agent teams chose for themselves, with 16 idle agents added. The rule was obeyed throughout, and no second crew formed. On every seed, the handoff loss rose from 2.7 to 5.2 rounds per layer (Figure 3b) and T rose from 37–39 to 49–56, back to the level of plain 16-agent teams (44–58). On two seeds the join's solver again read 15 answers from the store and lost 16 rounds, matching the join rule above. *Idle agents that are not allowed to form a crew buy nothing.*

6 Pre-registered Predictions

Each prediction was committed to the repository before its runs [13]. Table 3 lists all four.

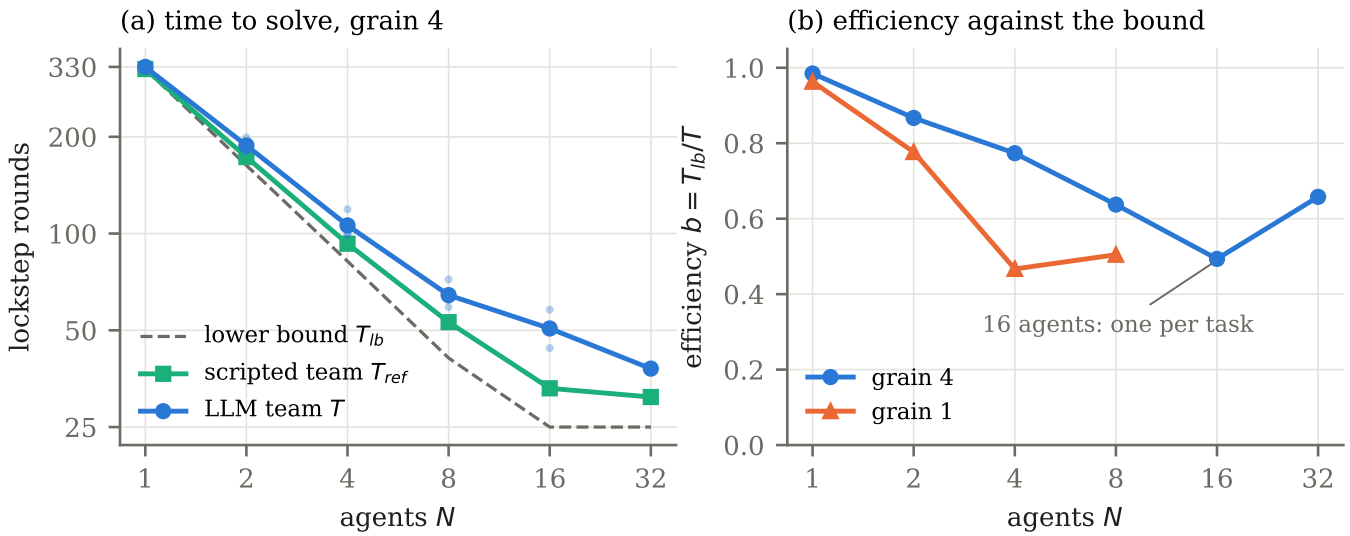


Figure 1: Scaling on the 16×4 problem. (a) Rounds to solve at grain 4 (log–log). Small dots are seeds; the line is their mean. The LLM team tracks the bound up to 4 agents and then levels off above the scripted team. 32 agents are faster than 16 although both have the same bound. (b) Efficiency $b = T_{\mathrm{lb}}/T$. Coarser tasks (grain 4) scale better than fine ones (grain 1).

Table 3: All predictions, written before their runs.

prediction	outcome
BSP: $T(32) = 46$ (42–58)	failed: 38
Layered BSP: $T(16) = 40$ (39–50)	failed: 50.7
Width 8: two per task lose less per layer, faster on every seed	held 3/3; both ≈3 rounds faster than predicted
No second crew: handoff loss ≈5, T rises	held 3/3

Why the scaling laws failed. The BSP reading models time as $T = \text{base} + S \cdot L$, a fixed loss L per superstep. Its first base, the Brent bound, spreads the final task and each layer's remainder over the whole team, which a layer-by-layer team cannot do. That error inflated L. Correcting the base after the fact still failed on the next, pre-registered size. The USL fit came within 1 round at 32 agents, but missed 16 agents by 4.7 rounds. Neither form can capture the result that matters most: whether a team has spare agents to form a second crew. The mechanistic predictions, which name a cause and the traces that should show it, held every time.

7 Discussion and Limitations

What carries over. In any pipeline of dependent steps, an agent that both produces an output and starts the next step serializes the handoff. A team gains when some agents prepare while others compute. LLM teams discovered this pipelining unprompted, but only when they had spare agents. This suggests a design rule for agent systems: give a team more agents than the widest stage, or assign the second crew explicitly.

Limitations.

- **One model.** All results so far come from one closed model. [TODO: open-weight replication.]
- **Three seeds.** Each setting has three seeds, and 16-agent runs vary widely (44–58 rounds). [TODO: 5–10 seeds for key settings.]
- **Synthetic task.** The task is synthetic by design, so that the bound is exact. [TODO: one real task with explicit dependencies.]
- **Confounded intervention.** The intervention also changed communication: fewer messages (12–55 against 134–182) and more store reads. Its control is earlier runs rather than a concurrent randomized arm. [TODO: concurrent rerun of plain 32.]
- **Idealized time.** Lockstep rounds idealize time. [TODO: asynchronous timing check.]

8 Conclusion

We introduced an environment where the best possible time of a team of LLM agents is known exactly, and used it to locate where the remaining time goes. Teams scale to an 8.7× speed-up, but they lose most of their time at handoffs and at the join. With spare agents, they recover part of it by forming crews on their own. Pre-registered mechanistic predictions held, while curve-fitting scaling laws did not.

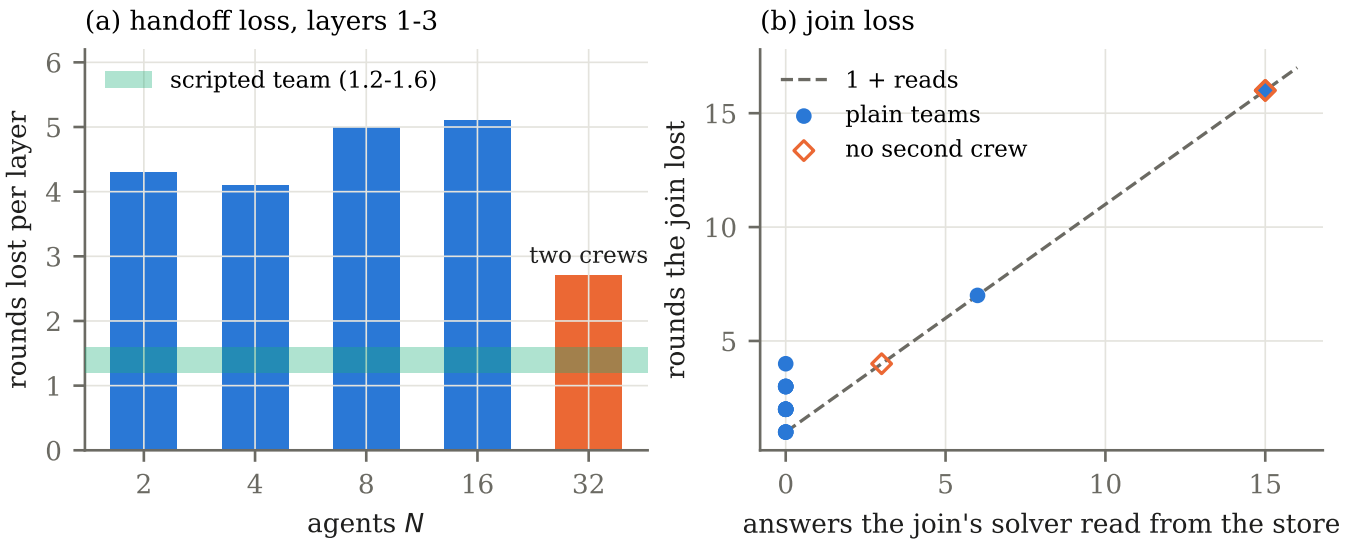


Figure 2: Where the time goes (grain 4, $w = 16$). (a) Mean handoff loss over layers 1–3. Teams of 2–16 lose 4–5 rounds per layer, against 1.2–1.6 for scripted teams. 32-agent teams, which split into two crews, lose 2.7. (b) Each point is one run. The join loses 1–4 rounds, plus about one round for every answer its solver reads from the store one key at a time. Diamonds are the no-second-crew intervention (§5.3).

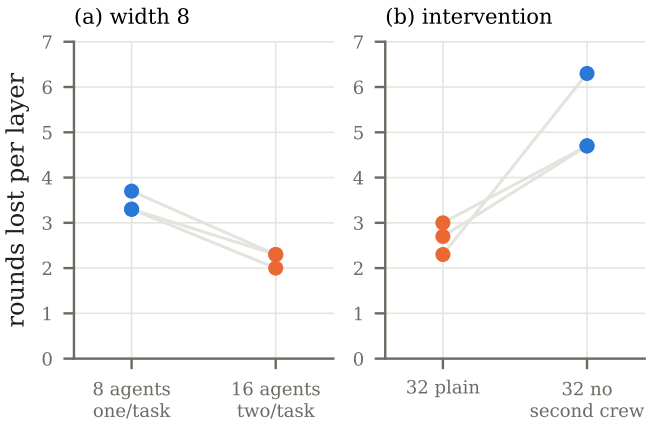


Figure 3: Two pre-registered tests of the crew mechanism, paired by seed. (a) Out of sample, on an 8-wide problem: 16 agents (two per task) formed two crews on every seed and lost less per layer than 8 agents. (b) Intervention: forbidding the second crew in 32-agent teams raised the handoff loss on every seed.

Reproducibility. Code, configurations, complete model-call archives, audit scripts and token accounting for every run will be released. An interactive replay visualizes each run as a town of agents, with a live task graph and T against T_{lb} [TODO: link to anonymized demo].

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[2] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.

[3] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[4] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.

[6] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17 (2):416–429, 1969.

[7] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.

[8] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *International Conference on Machine Learning (ICML)*, 2024.

[10] Yubin Kim et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025.

[11] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need. *Transactions on Machine Learning Research*, 2024.

[13] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

[14] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[15] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[16] Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large language model-based multi-agent collaboration. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[18] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitsin, Denis Kuznedelev, and Dan Alistarh. Hogwild! inference: Parallel LLM generation via concurrent attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[19] Leslie G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990.

[20] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[21] Zihan Xu, Haolin Tian, and Hai Jiang. A two-tier perspective on inference-time parallelism in multi-agent LLM systems. *arXiv preprint arXiv:2608.05791*, 2026.

[22] Zhen Yang, Xiaogang Xu, Wen Wang, Cong Chen, Xander Xu, and Ying-Cong Chen. Streaming communication in multi-agent reasoning. *arXiv preprint arXiv:2606.05158*, 2026.

[23] Junwei Yu, Yepeng Ding, and Hiroyuki Sato. DynTaskMAS: A dynamic task graph-driven framework for asynchronous and parallel LLM-based multi-agent systems. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2025.

[24] Enhao Zhang, Erkang Zhu, Gagan Bansal, Adam Fourney, Hussein Mozannar, and Jack Gerrits. Optimizing sequential multi-step tasks with parallel LLM agents. *arXiv preprint arXiv:2507.08944*, 2025.

[25] Yunheng Zou et al. El agente: An autonomous agent for quantum chemistry. *arXiv preprint arXiv:2505.02484*, 2025.

A Abandoned Versions

- **Symbolic recurrences.** Each task was a recurrence checked by SymPy. A strong model wrote the whole DAG as one closed-form expression without calling a tool, so neither work nor span held. This motivated hidden keyed functions.
- **Early hash version.** A grain-4 chain repeated a value and was solved in 324 actions against a bound of 325. Keys are now chosen so that no chain repeats a value.

B Token Usage

experiment	tokens
grain 1, N=1–8	61.3M
grain 4, N=1–8	149.9M
grain 4, N=32	90.8M
grain 4, N=16	59.5M
width 8, N=8, 16	52.7M
no second crew, N=32	105.9M